

31/8/26

CS 340

Lecture 14

An Efficient Algorithm
to test equivalence
of DFA states

General Algorithm to Compute ' $\equiv$ ' (or ' $\neq$ ')

Input: $M = (Q, \Sigma, q_0, \delta, F)$

Output: A table of distinct states pairs with distinguishable states marked

for $p \in F$ and $q \in Q - F$ do mark (p, q) ;

for $(p, q) \in Q \times Q$ do {
 if (marked (p, q)) continue ;
 if ($\neg$ marked (p, q)) {
 for all $a \in \Sigma$ do
 if ($(\delta(p, a), \delta(q, a))$ is marked) {mark (p, q) ; break}
 }
}

if (marked (p, q)) { Mark and remove all the pairs in List (p, q) recursively. That is, mark all the pairs in the list of (r, s) which gets marked at this step etc. }

// this may be implemented efficiently in
// ~~using~~ the steps $O(\text{no. of pairs marked})$
// using queue or stack.

if ($\neg$ mark (p, q)) {
 for $a \in \Sigma$ do
 if ($\delta(p, a) \neq \delta(q, a)$) { Add (p, q) to list of $(\delta(p, a), \delta(q, a))$ }
}
// end for

~~end~~ // (outermost) for

[with each unmarked pair (p, q) we have a list of pairs s.t. if (p, q) is marked then all elements in the list can be marked. Initially these lists are empty.]

Correctness

Claim: If $p \neq q$ then algorithm executes $\text{mark}(p, q)$.

Proof:

We show by induction on $|w|$ that ~~if~~

$$\delta(p, w) \in F \wedge \delta(q, w) \notin F \Rightarrow \text{mark}(p, q)$$

Base case:

$$w = \epsilon$$

$$\Rightarrow p \in F \wedge q \notin F$$

$\Rightarrow (p, q)$ is marked in the first for loop of the algo.

Induction step:

$$w = aw_1$$

$$\delta(p, aw_1) \in F \text{ and } \delta(q, aw_1) \notin F$$

$$\Rightarrow \delta(\delta(p, a), w_1) \in F \text{ and } \delta(\delta(q, a), w_1) \notin F$$

By I.H.

$(\delta(p, a), \delta(q, a))$ get ~~marked~~ marked by the algorithm.

Consider when pair (p, q) is considered. If $(\delta(p, a), \delta(q, a))$ is already marked then (p, q) gets marked in the second for loop.

~~If (p, q) is~~

If (p, q) does not get marked in this iteration, then it gets added to the list of $(\delta(p, a), \delta(q, a))$. Later when $(\delta(p, a), \delta(q, a))$ is marked, (p, q) also gets marked, in the second last 'statement' of the main for loop of the algorithm.

□

Complexity:

Let $|Q| = n$.

The first 'for-loop' of the algo works in $O(n^2)$ time.

The main 'for-loop' has $O(n^2)$ iterations. The first 'if' in this loop takes constant time, second and the fourth ~~to~~ 'if' take $O(|\Sigma|)$ time.

For the 'third ~~to~~ if' we do cumulative analysis (over all iterations of the for loop). As stated the time for this 'if' statement is $O(\text{number of pairs handled})$.

Each pair gets added to at most $|\Sigma|$ lists.

Therefore, there are at most $|\Sigma| n^2$ pair occurrences to remove.

Summing up, overall time of the algo is $O(|\Sigma| n^2)$.

Exercise:

In correctness proof, we showed that all distinguishable state pairs get marked. We also need to show that only such pairs get marked. This is easy to show. Technically, you may show this by induction on the no. of iterations of the 'main for loop'. You also need to ^{auxiliary} prove ~~proper~~ a relevant property of lists maintained for unmarked pairs of states.

We now point out how quotient (or minimum) automaton may be implemented from the state-equivalence table

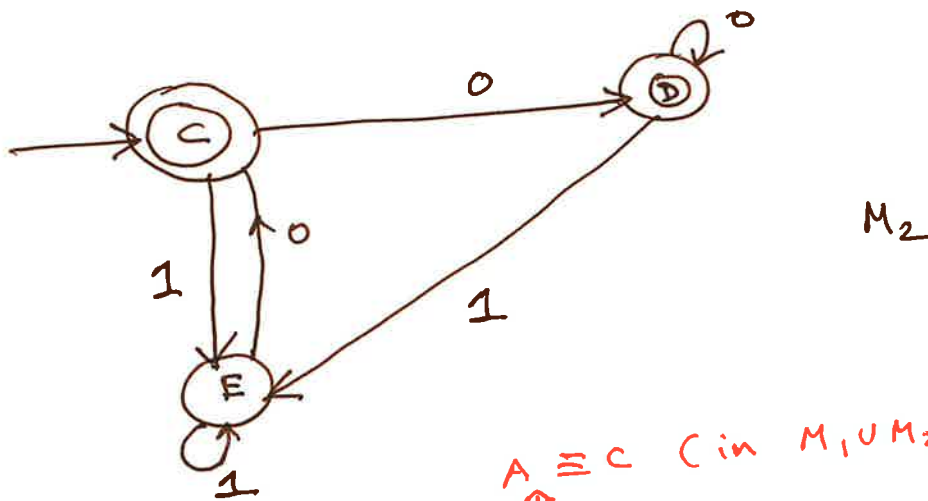
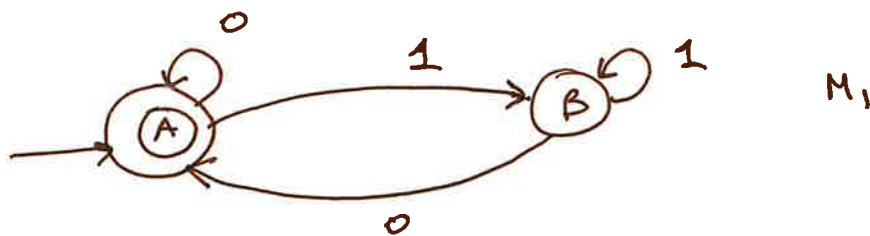
This is naturally done using ^{disjoint set} union-find algorithm.

To obtain partition of set of states as sets, we start with the family of sets where each set is a singleton state. ~~at~~ For each unmarked (p, q) in the table, we do $\text{find}(p)$, $\text{find}(q)$ and merge the two sets.

To construct the transition fn, ~~also we~~ $\delta_{M'}(\overline{[P]}, a)$ we use canonical names for sets as in union find algo. Assuming $[P]$ is such a canonical name, canonical name for $\delta_{M'}([P], a)$ is found by a 'find' operation on $\delta_M(P, a)$.

There may be at most $O(n^2)$ find operations. Time for them is $O(n^2 \log^* n)$. Therefore the minimization algorithm runs in overall complexity of $O((|Z| + \log^* n) n^2)$.

Testing Equivalence of DFA's



$A \equiv C$ (in $M_1 \cup M_2$)

$\Downarrow$

$$\forall z \in \Sigma^* \left[\delta(A, z) \in F \iff \delta(C, z) \in F \right]$$

$\Downarrow$

$$L(M_1) = L(M_2)$$

$(F = \{A, C, D\})$

List (A, C) : ~~(A, D)~~ (B, E)

$(A, D) \xrightarrow{0} (A, D)$
 $(A, D) \xrightarrow{1} (B, E)$

B	X			
C		X		
D		X		
E	X		X	X
	A	B	C	D

$(A, C) \xrightarrow{0} (A, D)$
 $(A, C) \xrightarrow{1} (B, E)$

$(C, D) \xrightarrow{0} (D, D)$
 $(C, D) \xrightarrow{1} (E, E)$

$(B, E) \xrightarrow{0} (A, C)$
 $(B, E) \xrightarrow{1} (B, E)$

List (A, D) : (A, D), (A, C) ~~(A, E)~~

List (B, E) : (A, D), (A, C), (B, E) ~~(B, D), (C, E), (D, E)~~

No more pairs may be marked. $A \equiv C$.